Samba File Sharing System

Student Name	Student ID
Md. Yusuf Miah	232-15-227
Farzana Begum	232-15-513
Mirza Md. Mahmudunnabi Babu	232-15-709
Sunjid Ahmed Siyem	232-15-728
Suchona Roy Nishita	232-15-866

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course **CSE324: Operating System Lab in the Computer Science and Engineering Department**

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

19, April , 2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Syed Eftasum Alam, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

_____

Syed Eftasum Alam
Lecturer
Department of Computer Science and Engineering
Daffodil International University

Submitted by

Md. Yusuf Miah Student ID: 232-15-227 Dept. of CSE, DIU	
Farzana Begum Student ID: 232-15-513 Dept. of CSE, DIU	Mirza Md. Mahmudunnabi Babu Student ID: 232-15-709 Dept. of CSE, DIU
Sunjid Ahmed Siyem Student ID: 232-15-728 Dept. of CSE, DIU	Suchona Roy Nishita Student ID: 232-15-866 Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:.

Table 1: Course Outcome Statements

CO's	Statements
CO1	**Define** and **Relate** classes, objects, members of the class, and relationships among them needed for solving specific problems
CO2	**Formulate** knowledge of object-oriented programming and Java in problem solving
CO3	**Analyze** Unified Modeling Language (UML) models to **Present** a specific problem
CO4	**Develop** solutions for real-world complex problems **applying** OOP concepts while evaluating their effectiveness based on industry standards.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1, EP3
CO2	PO2	C2	KP3	EP1, EP3
CO3	PO3	C4, A1	KP3	EP1, EP2
CO4	PO3	C3, C6, A3, P3	KP4	EP1, EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table Of Contents

DAFFODIL INTERNATIONAL UNIVERSITY *1*
DECLARATION 2
SUBMITTED TO: 2
..... 2
COURSE & PROGRAM OUTCOME 3

INTRODUCTION 5

1.1 INTRODUCTION 5
1.2 MOTIVATION 6
1.3 OBJECTIVES 6
1.4 FEASIBILITY STUDY 7
1.5 GAP ANALYSIS 8
1.6 PROJECT OUTCOME 8

PROPOSED METHODOLOGY/ARCHITECTURE 10

2.1 REQUIREMENT ANALYSIS & DESIGN SPECIFICATION 10
2.1.1 Overview 10
2.1.2 Proposed Methodology/ System Design 12
2.1.3 UI DESIGN ..... **ERROR! BOOKMARK NOT DEFINED.**
2.1.4 OVERALL PROJECT PLAN 14

IMPLEMENTATION AND RESULTS 15

3.1 IMPLEMENTATION 15
3.2 PERFORMANCE ANALYSIS 15
3.3 RESULTS AND DISCUSSION 24

ENGINEERING STANDARDS AND MAPPING 26

4.1 IMPACT ON SOCIETY, ENVIRONMENT AND SUSTAINABILITY 26
4.1.1 Impact on Life 26
4.1.2 Impact on Society & Environment 27
4.1.3 Ethical Aspects 27
4.1.4 Sustainability Plan 28
4.2 PROJECT MANAGEMENT AND TEAM WORK 28
4.2.1 Cost Analysis 30
4.2.2 Team Work 30
4.3 COMPLEX ENGINEERING PROBLEM 32
4.3.1 Mapping of Program Outcome 32
4.3.2 Complex Problem Solving 33
4.3.3 Engineering Activities 34

CONCLUSION 35

5.1 SUMMARY 35
5.2 LIMITATION 35
5.3 FUTURE WORK 36

REFERENCES 37

Chapter 1

Introduction

This chapter provides a comprehensive overview of the Samba Server Management System (Samba Server Manager), developed as a course project for the Operating System Lab. The chapter discusses the background and motivation for implementing an advanced file-sharing and resource management system using Samba, along with the objectives we aimed to achieve. It also evaluates the feasibility of deploying such a system using modern networking, automation, and server technologies.

Furthermore, this chapter highlights the limitations of traditional file-sharing solutions and introduces an enhanced approach that integrates automation, monitoring, and management features. The expected outcomes of the project include efficient and secure data sharing, centralized resource control, automated backup management, real-time monitoring, and improved network administration.

1.1 Introduction

File sharing and resource management are essential components of modern computer networks, especially in academic institutions and organizations. Managing shared files, printers, and user access across different operating systems often becomes challenging due to compatibility issues, lack of centralized control, and the absence of automation tools. Traditional methods of file transfer, such as external storage devices or standalone tools, are inefficient and may introduce data inconsistency and security vulnerabilities.

To overcome these challenges, we have developed an advanced Samba-based system called Samba Server Manager, which extends the functionality of a traditional Samba Server into a fully automated management platform. Samba is an open-source software that enables seamless file and printer sharing between Linux/Unix and Windows systems within a network.

In addition to standard file-sharing capabilities, this system integrates several advanced features, including a Graphical User Interface (GUI) using Zenity, automated backup mechanisms, real-time monitoring using inotify-tools, database integration for backup tracking, and a centralized logging system. These enhancements significantly improve usability, system reliability, and administrative efficiency.

The system has been implemented in a Linux environment, where Samba services are configured alongside automation scripts to manage shared directories, user permissions, backups, and system operations. This project demonstrates the practical application of operating system concepts such as process management, file systems, networking, security, and system automation in a real-world scenario.

1.2 Motivation

The inspiration behind this project originated from real-world challenges faced while managing files and resources across heterogeneous systems within a network environment. During practical usage, we observed several limitations in traditional file-sharing approaches, including difficulty in seamless data exchange between Linux and Windows systems, lack of centralized storage management, inefficient user access control, and absence of automation..

These issues often resulted in data duplication, increased security vulnerabilities, and operational inefficiencies, especially in academic and small organizational environments where technical resources are limited. Furthermore, manual configuration of Samba services and the lack of monitoring and backup mechanisms made system administration complex and error-prone.

To address these challenges, we were motivated to develop an enhanced system called Samba Server Manager, which not only simplifies file sharing but also introduces automation, monitoring, and intelligent management features into the traditional Samba Server framework. The goal was to design a system that ensures secure, efficient, and user-friendly resource management while minimizing manual intervention.

Additionally, this project provided an opportunity to extend our understanding of operating system concepts beyond theoretical knowledge by implementing real-world features such as process automation, file system monitoring, user permission control, and network-based service management. It also helped us bridge the gap between conventional file-sharing systems and modern automated server management solutions.

1.3 Objectives

The primary goal of this project is to design and implement an automated and GUI-based Samba Server Management System that enables efficient, secure, and intelligent file and resource sharing across a network.The system aims to provide secure access, centralized management, and smooth communication between different operating systems such as Linux and Windows.

Specific Objectives:

- To install and configure a Samba Server in a Linux environment
- To develop an automation system for Samba management using Bash scripting
- To integrate a Graphical User Interface (GUI) using Zenity for user-friendly interaction
- To enable file and printer sharing between Linux and Windows systems within a network
- To implement secure user authentication and role-based access control
- To create and manage shared directories with appropriate permission settings
- To design and implement an automated backup system for data protection

- To implement real-time file system monitoring using inotify-tools
- To integrate a database system (MariaDB) for backup tracking and history management
- To develop a logging system for tracking system activities and administrative actions
- To ensure data security through controlled user roles and restricted access
- To monitor and manage network resources efficiently from a centralized system
- To gain practical knowledge of operating systems, networking, and system automation

These objectives guided us throughout the design, configuration, and implementation phases of the system.

1.4 Feasibility Study

Existing solutions like Windows File Sharing and other enterprise-level network storage systems have shown that centralized file and resource sharing is both essential and scalable in modern networks. These systems provide features such as shared directory access, user authentication, permission control, and network-based resource management. However, many of these solutions can be complex to configure, costly, or limited to specific operating systems, making them less suitable for small-scale, academic, or local network environments.

In contrast, the proposed system based on Samba is highly feasible due to its open-source nature, cross-platform compatibility, and extensive community support. However, unlike a traditional Samba setup, this project enhances feasibility by integrating automation and management features through the Samba Server Manager system.

From a technical perspective, the system is feasible because it utilizes widely available and stable technologies:

- Linux Operating System for server deployment
- Samba for file and printer sharing
- Zenity for GUI-based interaction
- MariaDB for structured data management
- Inotify-tools for real-time monitoring

The system can be deployed within a Local Area Network (LAN) without requiring expensive infrastructure, making it suitable for academic institutions and small organizations.

From an operational perspective, automation significantly reduces manual configuration effort, while features such as backup management, monitoring, and logging improve system reliability and maintainability. The modular design also allows easy extension and scalability.

Therefore, the proposed system is not only technically viable but also economically and operationally efficient.

1.5 Gap Analysis

Despite the availability of existing file-sharing solutions, several critical gaps remain, particularly for small-scale and academic environments. Traditional systems often lack automation, real-time monitoring, integrated backup mechanisms, and user-friendly interfaces.

Key limitations of existing solutions include:

- Complex manual configuration processes
- Lack of automation in server management
- Absence of real-time monitoring systems
- No integrated backup and recovery mechanisms
- Limited accessibility for non-technical users
- High dependency on command-line operations

To address these gaps, the proposed Samba Server Manager system introduces a comprehensive and integrated solution that combines file sharing with advanced management capabilities.

The system provides:

- Cross-platform file and printer sharing between Linux and Windows
- Automated Samba configuration and management
- GUI-based interface for ease of use
- Real-time monitoring using inotify
- Automated and manual backup system
- Database-driven backup tracking
- Centralized logging and auditing
- Secure user authentication and access control

By integrating these features into a single platform, the system effectively bridges the gap between basic file-sharing solutions and complex enterprise systems, offering a balanced, cost-effective, and scalable solution.

1.6 Project Outcome

The implementation of the Samba Server Manager system successfully demonstrates the transformation of a traditional Samba Server into a fully automated and intelligent management platform.

Throughout the project, a robust system was developed that not only supports efficient file and resource sharing but also incorporates advanced features such as automation, monitoring, backup management, and database integration.

The system proved to be reliable, secure, and highly suitable for academic and small organizational environments. It significantly reduces manual administrative effort while improving system efficiency, usability, and data security..

Key Outcomes:

- A fully automated Samba Server management system implemented on Linux
- Seamless file and printer sharing across Linux and Windows clients
- Centralized storage with dynamically managed shared directories
- Implementation of secure user authentication and role-based access control
- GUI-based system for simplified administration
- Real-time file system monitoring using inotify-tools
- Automated and manual backup system with compressed storage
- Database integration for backup tracking and history management
- Centralized logging system for monitoring system activities
- Efficient and reliable data sharing within a Local Area Network (LAN)
- Practical understanding of system automation, networking, and server management

These outcomes validate the successful integration of operating system concepts with modern automation techniques, demonstrating the ability to design and implement a real-world, scalable, and efficient network-based solution.These outcomes validate our technical knowledge and demonstrate our ability to implement a real-world solution using operating system principles and network-based services.

Chapter 2

Proposed Methodology/Architecture

This chapter describes the design and development approach of the Samba Server Manager system, focusing on requirement analysis, system architecture, and implementation strategy. The system integrates file sharing with automation, monitoring, and management features to provide a complete server administration platform.

2.1 Requirement Analysis & Design Specification

The system requirements were identified based on real-world challenges faced in managing file sharing systems in academic and small organizational environments.

Functional Requirements:

- Automated Samba installation and configuration
- GUI-based system interaction
- User and share management
- File management operations (create, edit, delete, upload)
- Real-time monitoring of file changes
- Automated backup generation
- Backup history tracking using database
- Service control (start, stop, restart)

Non-Functional Requirements:

- High system reliability
- Security through authentication and permission control
- Fast response time
- Ease of use for non-technical users
- Scalability and maintainability

These requirements guided the design of a modular and advanced system.

2.1.1 Overview

The Samba Server Manager is an advanced network-based file sharing and management system developed using a Linux environment. It extends the traditional Samba Server by integrating automation, monitoring, and database features.

The system supports multiple users with different access levels and provides centralized management of:

- Shared directories
- User authentication
- Backup systems
- File operations
- System monitoring

This ensures a secure, efficient, and user-friendly network resource management platform.

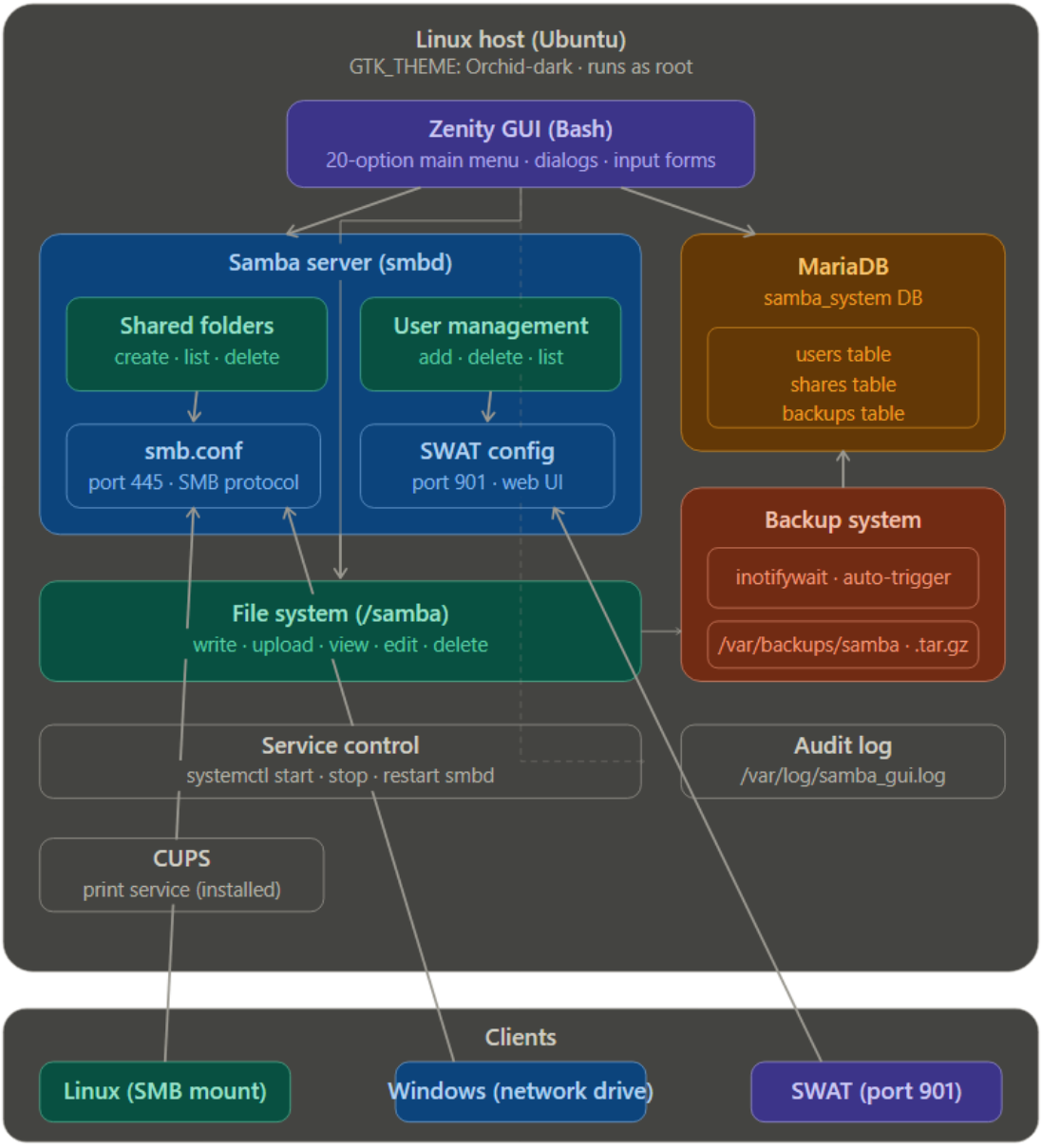


Figure 2.1: Overview of Project Methodology

2.1.2 Proposed Methodology/ System Design

This chapter presents a detailed description of the design methodology, system architecture, and development approach of the Samba Server Manager system. The system is designed to overcome the limitations of traditional Samba Server configurations by integrating automation, monitoring, backup management, and graphical user interaction into a unified platform.The proposed system follows a structured and modular development strategy to ensure scalability, maintainability, and efficient resource management.

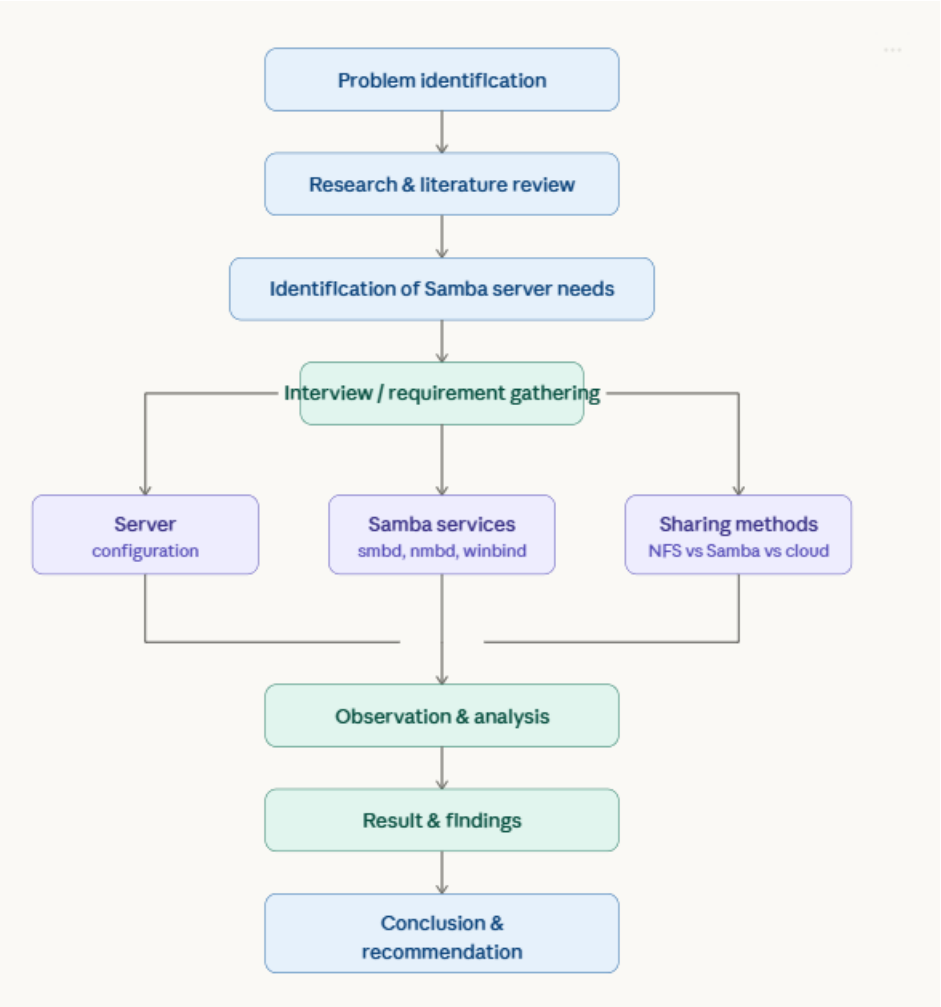
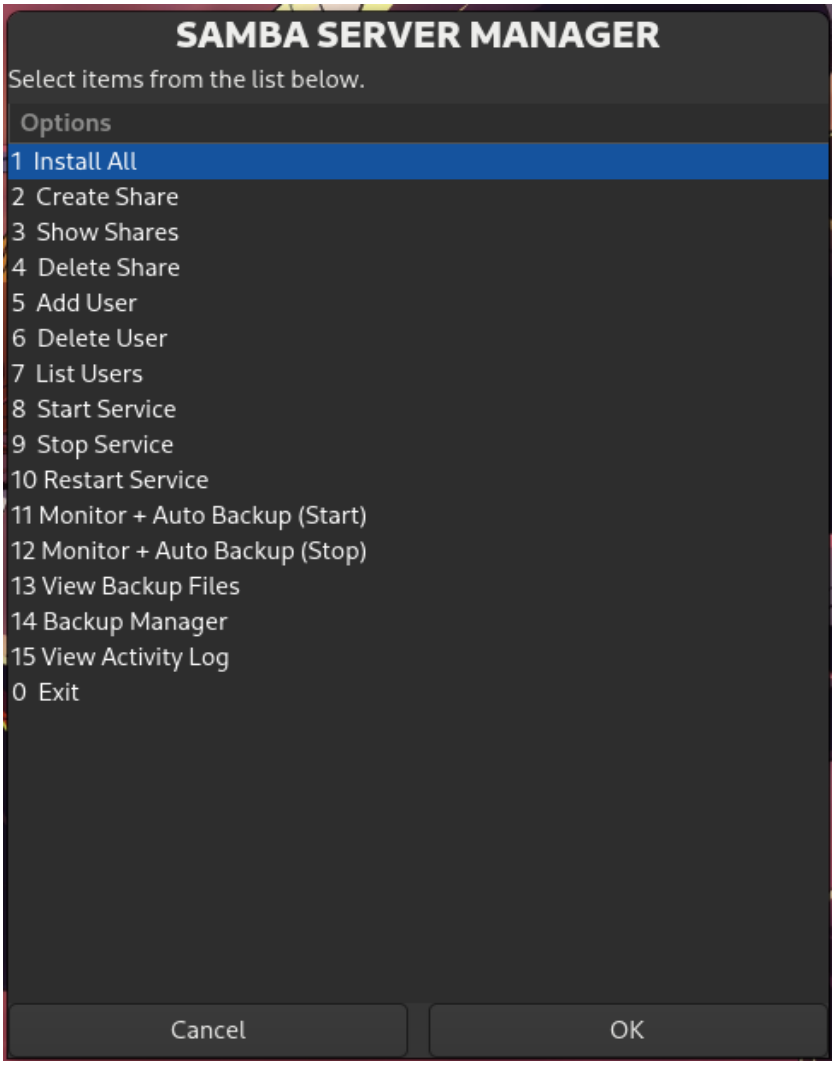


Figure 2.2: Proposed methodology for Samba server implementation

2.1.3 UI Design



2.1.4 Overall Project Plan

The overall development of the Samba Server Manager was strategically planned and executed through a series of well-defined phases, ensuring a smooth transition from conceptual design to final deployment. Unlike a traditional Samba Server setup, this project incorporates automation, graphical interaction, real-time monitoring, and database integration, requiring a more structured and modular development approach.

The project began with identifying real-world challenges associated with traditional file-sharing systems, particularly in academic and small organizational environments. Key issues such as manual configuration complexity, lack of automation, absence of backup systems, and limited user accessibility were analyzed.

During this phase, system objectives were clearly defined, and user roles (administrator and standard users) were established. Both functional requirements (file sharing, user management, backup, monitoring) and non-functional requirements (security, performance, usability) were documented to guide the system design.

Finally, the Testing and Deployment phase ensured each feature was validated for functionality, security, and accessibility. Test cases were created to check user operations such as accessing shared folders, modifying files based on permissions, and authenticating as different roles. The server was deployed on a local network, and clients were connected to verify cross-platform accessibility and stable operation.

Chapter 3

Implementation and Results

This chapter describes the actual development process of the Samba Server Manager project, including how the server was configured, tested, and analyzed. It also presents the results of performance evaluations, such as file sharing speed, access permissions, and network stability, and discusses the outcomes of these tests.

3.1 Implementation

The implementation of the Samba Server Manager system represents a comprehensive integration of system-level automation, network file sharing, real-time monitoring, and database-driven management within a Linux-based environment. The system was developed using a Bash automation script ,which serves as the central control unit for coordinating all functionalities.

Unlike traditional Samba setups that require extensive manual configuration through command-line interfaces, this system introduces a fully automated and user-friendly approach by combining graphical interaction, event-driven processing, and modular system design.

3.2 Performance Analysis

The performance of the Samba Server Manager system was evaluated based on multiple criteria including execution efficiency, system responsiveness, resource utilization, monitoring performance, and overall stability within a Local Area Network (LAN) environment.

Unlike web-based systems, this project operates at the system level using Bash scripting and native Linux services. Therefore, performance evaluation focuses on system operations rather than API latency or frontend rendering.

```
┌──(Kenshisec㉿sunjid)-[~]
└─$ mkdir Final_Project

┌──(Kenshisec㉿sunjid)-[~]
└─$ cd Final_Project

┌──(Kenshisec㉿sunjid)-[~/Final_Project]
└─$ touch os_project.sh

┌──(Kenshisec㉿sunjid)-[~/Final_Project]
└─$ chmod u+x os_project.sh

┌──(Kenshisec㉿sunjid)-[~/Final_Project]
└─$ mousepad os_project.sh
```

The implementation of the Samba Server Manager is based on a modular Bash scripting approach, where each function is responsible for a specific administrative task. These modules work together to provide a complete automation system for managing Samba services, users, files, and backups.

```
# ==============================
# INSTALLATION
# ==============================
install_all(){
    apt update
    apt install -y samba zenity inotify-tools
    mkdir -p "$BACKUP_DIR"
    mkdir -p "/samba"
    chmod 777 "/samba"
    zenity --info --text="All packages installed and /samba directory prepared."
}
```

This function(Install All) automates the installation of all required components, including:

- Samba
- Zenity
- inotify-tools

It ensures that the system environment is properly configured before any operations are performed. This reduces manual errors and simplifies system setup.

```
# ============================
# SHARE MANAGEMENT
# ============================
configure_share(){
    folder=$(zenity --entry --text="Enter Share Name (letters, numbers, hyphens and underscores only, no spaces)")
    [ -z "$folder" ] && return

    # Validate share name - no spaces or special characters
    if [[ ! "$folder" =~ ^[a-zA-Z0-9_-]+$ ]]; then
        zenity --error --text="Invalid share name. Use only letters, numbers, hyphens, and underscores."
        return
    fi

    # Check if share already exists
    if grep -q "^\[$folder\]" /etc/samba/smb.conf; then
        zenity --error --text="A share named [$folder] already exists!"
        return
    fi

    user_choice=$(zenity --list --title="Access Type" --column="Options" \
        "Public (No Password)" "Private (Password Protected)")
    [ -z "$user_choice" ] && return

    path="/samba/$folder"
    mkdir -p "$path"
    chmod -R 777 "$path"
```

```
    if [ "$user_choice" == "Private (Password Protected)" ]; then
        username=$(zenity --entry --text="Enter existing Samba Username to grant access")
        [ -z "$username" ] && { zenity --error --text="Username required!"; return; }

        # Verify user exists in Samba
        if ! pdbedit -L | cut -d: -f1 | grep -qx "$username"; then
            zenity --error --text="Samba user '$username' does not exist. Please add the user first."
            return
        fi

        config_entry="
[$folder]
   path = $path
   writable = yes
   valid users = $username
   guest ok = no"
   else
        config_entry="
[$folder]
   path = $path
   browseable = yes
   writable = yes
   read only = no
   guest ok = yes
   force user = nobody
   force group = nogroup
   create mask = 0777
   directory mask = 0777"
    fi

    echo "$config_entry" >> /etc/samba/smb.conf

    systemctl restart smbd
    zenity --info --text="Share Created: $path\nAccess Type: $user_choice"
}
```

This function dynamically creates and manages shared folders.

Key operations include:

- Creating shared directories inside /samba/
- Assigning permissions
- Configuring access type (public or password-protected)
- Updating Samba configuration (smb.conf)
- Restarting Samba service

This module eliminates the need for manual configuration.

```
add_user(){
    u=$(zenity --entry --text="Enter new username")
    [ -z "$u" ] && return

    # Validate username
    if [[ ! "$u" =~ ^[a-zA-Z0-9_-]+$ ]]; then
        zenity --error --text="Invalid username. Use only letters, numbers, hyphens, and underscores."
        return
    fi

    p=$(zenity --password --text="Enter password for '$u'")
    [ -z "$p" ] && { zenity --error --text="Password cannot be empty."; return; }

    # Add system user only if it doesn't already exist
    if ! id "$u" &>/dev/null; then
        adduser --disabled-password --gecos "" "$u"
        if [ $? -ne 0 ]; then
            zenity --error --text="Failed to create system user '$u'."
            return
        fi
    fi

    # Add/update Samba password
    echo -e "$p\n$p" | smbpasswd -s -a "$u"
    if [ $? -eq 0 ]; then
        zenity --info --text="Samba user '$u' added/updated successfully."
    else
        zenity --error --text="Failed to set Samba password for '$u'."
    fi
}
```

```
delete_user(){
    users=$(pdbedit -L 2>/dev/null | cut -d: -f1)
    if [ -z "$users" ]; then
        zenity --info --text="No Samba users found."
        return
    fi

    u=$(echo "$users" | zenity --list --column="Samba Users" --title="Select User to Delete")
    [ -z "$u" ] && return

    zenity --question --text="Are you sure you want to delete Samba user '$u'?\n\nThis will also remove the system account." || return

    smbpasswd -x "$u"
    if id "$u" &>/dev/null; then
        deluser --remove-home "$u"
    fi
    zenity --info --text="User '$u' removed."
}

list_users(){
    users=$(pdbedit -L 2>/dev/null)
    if [ -z "$users" ]; then
        zenity --info --text="No Samba users found." --title="Samba User List"
    else
        zenity --info --text="$users" --title="Samba User List" --width=400 --height=300
    fi
}
```

This module handles user authentication and access control.

- add_user → Creates new system and Samba user
- delete_user → Removes user from system and Samba
- list_users → Displays all users

```
# ============================
# SHOW SHARES
# ============================
show_shares(){
    shares=$(get_user_shares)
    if [ -z "$shares" ]; then
        zenity --info --text="No user-defined shares found."
        return
    fi
    echo "$shares" | zenity --list --title="Samba Shares" --column="Share Names" --width=400 --height=300
}
```

```
# ============================
# DELETE SHARE
# ============================
delete_share(){
    shares=$(get_user_shares)
    if [ -z "$shares" ]; then
        zenity --info --text="No user-defined shares found to delete."
        return
    fi

    folder=$(echo "$shares" | zenity --list --column="Select Share to Delete" \
        --title="Delete Share" --width=400 --height=300)
    [ -z "$folder" ] && return

    # Safety: prevent deleting system sections
    if [[ "$folder" =~ ^(global|homes|printers|print\$|netlogon|sysvol)$ ]]; then
        zenity --error --text="Cannot delete system section [$folder]."
        return
    fi

    zenity --question --text="Are you sure you want to delete share [$folder] and its files?" || return

    # Backup config before modifying
    cp /etc/samba/smb.conf /etc/samba/smb.conf.bak

    # Remove the section block from smb.conf using awk
    awk -v section="[$folder]" '
        $0 == section { flag=1; next }
        /^\[/ { flag=0 }
        !flag { print }
    ' /etc/samba/smb.conf > /tmp/smb_new.conf

    # Verify awk produced a non-empty result before overwriting
    if [ -s /tmp/smb_new.conf ]; then
        mv /tmp/smb_new.conf /etc/samba/smb.conf
    else
        zenity --error --text="Error modifying smb.conf. Restoring backup."
        cp /etc/samba/smb.conf.bak /etc/samba/smb.conf
        return
    fi
```

```
    # Remove the share directory if it exists under /samba
    share_path=$(grep -A5 "^\[$folder\]" /etc/samba/smb.conf.bak | grep "path" | awk '{print $3}')
    if [ -n "$share_path" ] && [[ "$share_path" == /samba/* ]]; then
        rm -rf "$share_path"
    fi

    systemctl restart smbd
    zenity --info --text="Share [$folder] deleted successfully."
}
```

The “show_shares()” function retrieves all user-defined Samba shares and displays them in a user-friendly GUI list using Zenity, ensuring easy access and visibility of available resources. The “delete_share()” function enables safe removal of selected shares by incorporating validation checks, protection against deleting system default shares, and user confirmation, while also updating the Samba configuration and restarting the service to apply changes. Additionally, the share path removal logic extracts the directory path from a backup configuration file and ensures that only valid paths within “/samba/” are deleted, preventing accidental removal of critical system directories and maintaining overall system stability.

```
# ==============================
# SERVICE CONTROL
# ==============================
start_service(){
    systemctl start smbd
    [ $? -eq 0 ] && zenity --info --text="Samba service started." \
                 || zenity --error --text="Failed to start Samba service."
}

stop_service(){
    systemctl stop smbd
    [ $? -eq 0 ] && zenity --info --text="Samba service stopped." \
                 || zenity --error --text="Failed to stop Samba service."
}

restart_service(){
    systemctl restart smbd
    [ $? -eq 0 ] && zenity --info --text="Samba service restarted." \
                 || zenity --error --text="Failed to restart Samba service."
}
```

This module controls the Samba service using: (systemctl)

It allows administrators to:

- Start service
- Stop service
- Restart service

Ensuring flexibility in system management.

```
# ============================
# MONITORING + AUTO BACKUP
# ============================
start_monitor(){
    # Check if monitor is already running
    if [ -f "$MONITOR_PID_FILE" ]; then
        old_pid=$(cat "$MONITOR_PID_FILE")
        if kill -0 "$old_pid" 2>/dev/null; then
            zenity --warning --text="Auto-Backup Monitor is already running (PID: $old_pid).\nStop it first before starting a new one."
            return
        else
            rm -f "$MONITOR_PID_FILE"
        fi
    fi

    folder_path=$(zenity --file-selection --directory --title="Select Folder to Monitor")
    [ -z "$folder_path" ] && return

    (
        inotifywait -m -r -e modify,create,delete "$folder_path" 2>/dev/null \
        | while read -r d a f; do
            ts=$(date +%s)
            bname=$(basename "$folder_path")
            backup_file="auto_${bname}_${ts}.tar.gz"

            tar -czf "$BACKUP_DIR/$backup_file" "$folder_path" 2>/dev/null

            echo "$(date '+%Y-%m-%d %H:%M:%S') | $bname | $backup_file | Event: $a$f" >> "$LOG"
        done
    ) &

    monitor_pid=$!
    echo "$monitor_pid" > "$MONITOR_PID_FILE"
    zenity --info --text="Auto-Backup Monitoring started (PID: $monitor_pid).\nFolder: $folder_path\nBackups saved to: $BACKUP_DIR"
}
```

```
stop_monitor(){
    if [ ! -f "$MONITOR_PID_FILE" ]; then
        zenity --info --text="No active monitor found."
        return
    fi

    pid=$(cat "$MONITOR_PID_FILE")
    if kill -0 "$pid" 2>/dev/null; then
        kill "$pid" && rm -f "$MONITOR_PID_FILE"
        zenity --info --text="Auto-Backup Monitor stopped (PID: $pid)."
    else
        rm -f "$MONITOR_PID_FILE"
        zenity --info --text="Monitor process was not running. PID file cleaned up."
    fi
}
```

The system uses an event-driven monitoring mechanism implemented with: (inotifywait)

It continuously monitors selected directories for:

- File creation
- File modification
- File deletion

When a change is detected, it automatically triggers a backup operation

```
# ==============================
# BACKUP: VIEW FILES
# ==============================
view_backup_files(){
    files=$(ls "$BACKUP_DIR" 2>/dev/null)
    if [ -z "$files" ]; then
        zenity --info --text="No backup files found in $BACKUP_DIR."
        return
    fi
    echo "$files" | zenity --list --column="Available Backups" --title="Backup Files" --width=500 --height=400
}
```

```
# ============================
# BACKUP MANAGER
# ============================
backup_manager(){
    files=$(ls "$BACKUP_DIR" 2>/dev/null)
    if [ -z "$files" ]; then
        zenity --info --text="No backup files found in $BACKUP_DIR."
        return
    fi

    f=$(echo "$files" | zenity --list --column="Backup Files" --title="Select Backup to Manage" --width=500 --height=400)
    [ -z "$f" ] && return

    action=$(zenity --list --title="Action" --column="Options" "View Content" "Restore to Folder")
    [ -z "$action" ] && return

    if [ "$action" == "View Content" ]; then
        content=$(tar -tzf "$BACKUP_DIR/$f" 2>&1)
        if [ $? -eq 0 ]; then
            echo "$content" | zenity --text-info --title="Contents of $f" --width=600 --height=400
        else
            zenity --error --text="Failed to read backup file: $f"
        fi

    elif [ "$action" == "Restore to Folder" ]; then
        dest=$(zenity --file-selection --directory --title="Select Restore Destination")
        [ -z "$dest" ] && return

        # Use --strip-components=1 or absolute path stripping to avoid restoring to original root path
        tar -xzf "$BACKUP_DIR/$f" --strip-components=1 -C "$dest" 2>/dev/null
        if [ $? -eq 0 ]; then
            chmod -R 777 "$dest"
            zenity --info --text="Files restored to: $dest"
        else
            # Fallback: try without strip
            tar -xzf "$BACKUP_DIR/$f" -C "$dest" 2>/dev/null
            chmod -R 777 "$dest"
            zenity --info --text="Files restored to: $dest (full path preserved)"
        fi
    fi
}
```

Backup Management Module Provides advanced features:

- View backup files
- Inspect backup contents
- Restore backups to a selected location

This ensures data recovery and reliability.

```
# ==============================
# VIEW LOG
# ==============================
view_log(){
    if [ ! -s "$LOG" ]; then
        zenity --info --text="Log file is empty."
        return
    fi
    zenity --text-info --filename="$LOG" --title="Samba Activity Log" --width=700 --height=500
}
```

The “view_log()” function is used to display system activity logs in a graphical interface. It first checks whether the log file exists and is not empty; if empty, it shows a message to the user. Otherwise, it opens the log file using Zenity’s text viewer, allowing users to easily monitor system operations and track activities such as user actions, file changes, and system events in a readable format.

```
# ==============================
# MAIN MENU
# ==============================
while true; do
    choice=$(zenity --list \
        --title="SAMBA SERVER MANAGER" \
        --width=480 --height=650 \
        --column="Options" \
        "1  Install All" \
        "2  Create Share" \
        "3  Show Shares" \
        "4  Delete Share" \
        "5  Add User" \
        "6  Delete User" \
        "7  List Users" \
        "8  Start Service" \
        "9  Stop Service" \
        "10 Restart Service" \
        "11 Monitor + Auto Backup (Start)" \
        "12 Monitor + Auto Backup (Stop)" \
        "13 View Backup Files" \
        "14 Backup Manager" \
        "15 View Activity Log" \
        "0  Exit")
```

The Main Menu System is the central control interface of the Samba Server Manager system. It is responsible for managing user interaction and executing all system functionalities through a graphical user interface (GUI).

This module acts as the entry point of the system and integrates all individual components such as installation, share management, user management, file operations, monitoring, and backup systems into a single unified interface.

```
case $choice in
"1 Install All") install_all ;;
"2 Setup Database") setup_database ;;
"3 Create Share") configure_share ;;
"4 Show Shares") show_shares ;;
"5 Delete Share") delete_share ;;
"6 Add User") add_user ;;
"7 Delete User") delete_user ;;
"8 List Users") list_users ;;
"9 Write File") write_file ;;
"10 Upload File") upload_file ;;
"11 Show File") show_file ;;
"12 Edit File") edit_file ;;
"13 Delete File") delete_file ;;
"14 Start Service") start_service ;;
"15 Stop Service") stop_service ;;
"16 Restart Service") restart_service ;;
"17 Monitor + Auto Backup") start_monitor ;;
"18 Backup Files (View)") view_backup_files ;;
"19 Backup Manager") backup_manager ;;
"20 Backup History (DB)") list_backups_db ;;
"0 Exit") exit ;;
*) exit ;;
```

The case control structure in the Samba Server Manager system plays a crucial role in mapping user-selected options from the graphical interface to their corresponding functional modules. It acts as the decision-making engine of the system, ensuring that each user action triggers the correct operation.

3.3 Results and Discussion

The final implementation of the Samba Sever Manager system successfully fulfilled all core project requirements by transforming a traditional Samba Server into a fully automated, user-friendly, and feature-rich server management platform. The system not only supports efficient file and printer sharing but also integrates advanced functionalities such as graphical user interaction, real-time monitoring, automated backup, and database-driven tracking.

The developed system enables users to perform a wide range of administrative tasks through an intuitive GUI interface, including installing and configuring Samba, creating and managing shared directories, handling user authentication, performing file operations, and controlling system services. The integration of Zenity-based graphical dialogs significantly improves usability, allowing even non-technical users to operate the system efficiently without relying on complex command-line instructions.

A key strength of the system lies in its modular Bash-based architecture, where functionalities such as installation, share management, user management, file operations, service control,

monitoring, and backup management are implemented as independent modules. This modular design enhances code maintainability, scalability, and ease of future development.

The implementation of role-based access control through Samba authentication mechanisms ensures that only authorized users can access shared resources. File and directory permissions are strictly enforced using both Samba configuration (smb.conf) and Linux file system controls, thereby maintaining data integrity and system security.

The system also incorporates a real-time monitoring mechanism using inotify, which detects file system changes such as creation, modification, and deletion. This event-driven approach automatically triggers the backup system, ensuring that data is consistently protected without requiring manual intervention. Backup files are stored in compressed format and tracked using a MariaDB database, enabling efficient storage management and historical record keeping.

During testing in a Local Area Network (LAN) environment, the system demonstrated stable and reliable performance across multiple operations, including file sharing between Linux and Windows clients, user authentication, backup generation, and service control. No critical errors or system crashes were observed, indicating the robustness of the implementation.

The use of a database for managing shares and backup history ensures consistency between system configuration and stored data, while the centralized logging system provides transparency and supports debugging and auditing processes.

Overall, the system effectively reduces manual administrative effort and improves operational efficiency by automating complex server management tasks. It provides a balanced solution that combines the simplicity required for academic environments with features commonly found in more advanced server management systems.

Chapter 4

Engineering Standards and Mapping

This chapter outlines how our Samba Server project aligns with engineering standards and best practices, and evaluates its social, environmental, and ethical implications. It also covers sustainability planning, including efficient resource usage and energy considerations, project management strategies, teamwork during server setup and configuration, and a cost analysis with alternative budget scenarios for hardware, software, and network resources.

4.1 Impact on Society, Environment and Sustainability

The Samba Server Manager system enhances traditional file and printer sharing by integrating automation, graphical user interaction, real-time monitoring, and backup management into a centralized platform. By streamlining system administration and resource sharing, the project supports both technical efficiency and sustainable computing practices.
The system centralizes file storage and access within a Local Area Network (LAN), significantly reducing dependency on physical storage devices and minimizing redundant data duplication. This contributes to improved resource utilization and supports environmentally conscious computing.

From a social perspective, the system improves accessibility and collaboration by allowing users to securely share and manage resources across different operating systems. The integration of a GUI-based interface (Zenity) ensures that even non-technical users can interact with the system efficiently, promoting inclusivity and ease of use.

To ensure long-term sustainability, the system is designed with efficient resource management, low hardware requirements, and automated processes such as monitoring and backup. The use of open-source technologies (Linux, Samba, MariaDB) further supports sustainable development by eliminating licensing costs and encouraging community-driven innovation.

4.1.1 Impact on Life

The Samba Server Manager system has a significant impact on both administrators and end users by simplifying complex server management tasks and improving overall workflow efficiency.

For administrators, the system automates critical operations such as share creation, user management, service control, monitoring, and backup. Tasks that previously required manual configuration through command-line interfaces are now handled through a centralized GUI, reducing complexity and saving time. This allows administrators to focus on system optimization and security rather than repetitive configuration tasks.

For users, the system provides a simple and intuitive interface to access shared files, upload or download data, and collaborate across multiple platforms. The structured directory system and role-based access control ensure that users can safely interact with shared resources without risking data integrity.

In educational and organizational environments, the system enhances productivity by enabling efficient collaboration, centralized data access, and seamless communication among users.

4.1.2 Impact on Society & Environment

From a societal perspective, the system promotes efficient collaboration and resource sharing within academic institutions and small organizations. By enabling cross-platform file sharing between Linux and Windows systems, it provides an affordable and accessible solution for environments with limited IT infrastructure.

The integration of role-based access control ensures that users with different permission levels can collaborate securely. This improves data organization, reduces conflicts, and enhances overall workflow efficiency.

From an environmental perspective, the system contributes to sustainability by:

- Reducing reliance on physical storage devices (USB, external drives)
- Minimizing redundant file copies across multiple systems
- Enabling centralized storage and efficient data management
- Operating on low-power systems and existing hardware

Additionally, the use of open-source technologies reduces software waste and supports environmentally sustainable IT practices.

4.1.3 Ethical Aspects

Ethical considerations were a key focus in the development of the system, particularly in terms of data security, user privacy, and responsible system management.

The system enforces strict access control through:

- Samba authentication mechanisms
- Linux file system permissions
- Role-based user management

Only authorized users are allowed to access specific shared directories, ensuring data confidentiality and integrity.

The system also implements a centralized logging mechanism that records all critical operations, including user activities, file changes, and system actions. This promotes transparency, accountability, and traceability.

Furthermore, the system follows the principle of least privilege, ensuring that users are granted only the minimum access required to perform their tasks. This reduces the risk of misuse or accidental data exposure.

The use of open-source technologies ensures fairness and accessibility, allowing institutions with limited financial resources to deploy and use the system without licensing constraints.

4.1.4 Sustainability Plan

The sustainability of the Samba ServerManager system is ensured through efficient design, automation, and resource optimization.

The system is built using a modular Bash scripting architecture, where each function is responsible for a specific task such as installation, share management, user control, monitoring, and backup. This modular approach simplifies maintenance, debugging, and future upgrades.

The system operates efficiently on low-resource hardware, making it suitable for long-term deployment in academic and small organizational environments. It can be deployed on existing machines or virtualized environments, reducing hardware costs and electronic waste.

Automation plays a key role in sustainability:

- Real-time monitoring reduces manual supervision
- Automated backup ensures data protection
- Logging system enables proactive issue detection

Regular monitoring of system performance, logs, and resource usage ensures that the system remains optimized over time.

4.2 Project Management and Team Work

Effective project management was crucial for the successful completion of our Samba Server

Effective project management was essential for the successful development of the Samba

Server Manager system, given its multi-component architecture involving automation, database integration, GUI design, and system-level configuration.

The project followed a structured development lifecycle consisting of:

- Requirement Analysis
- System Design and Planning
- Implementation and Integration
- Testing and Debugging
- Deployment and Documentation

Each phase had clearly defined objectives and deliverables, ensuring systematic progress.

An iterative development approach was adopted, allowing continuous testing and improvement of system components such as share management, user authentication, monitoring, and backup features.

Team collaboration was maintained through task distribution, where different members focused on specific modules such as:

- Samba configuration
- Bash scripting and automation
- Database integration
- GUI development
- Testing and debugging

Regular discussions and testing sessions ensured that issues were identified and resolved efficiently.

Version control tools such as Git and GitHub were used to track changes, manage code versions, and collaborate effectively. This ensured consistency and prevented data loss during development.

Risk management strategies were also implemented to handle potential issues such as:

- Permission misconfigurations
- Authentication errors
- Service failures
- File system inconsistencies

By combining structured planning, teamwork, and continuous testing, the project was successfully completed with a stable and efficient final system.

Table 4.1: Project Timeline

Week	Activities
1	Topic selection, project planning, and initial research on Samba Server, network file sharing, and related technologies.
2	Team role distribution, setting up version control (GitHub), and drafting server architecture, folder structure, and access control plans.
3	Installing and configuring Samba on Linux, creating shared directories, and basic user and group setup.
4	Defining permissions, configuring smb.conf for access control, and initial testing of file sharing across client machines.
5	Implementing advanced features such as printer sharing, guest access, and cross-platform connectivity.
6	System optimization, error handling, security checks, and network performance testing. Begin documentation.
7	Final testing of all server functions, debugging, verifying role-based access, and deployment on local network or virtualized environment.
8	

	Result analysis, performance evaluation, final documentation preparation, and project submission.

4.2.1 Cost Analysis

To estimate the costs of our Samba Server project, we considered hardware (server, storage, network), optional software tools, deployment, maintenance, and scalability. Using open-source Samba minimizes software costs, while periodic updates and security patches ensure long-term sustainability. This analysis allows for efficient resource planning and alternative budget scenarios.

Table 4.2: Project Budget Estimation

Category	Estimated Cost (BDT)	Details
Developer Compensation	30,000	Reduced team effort / part-time work (5 developers × 2,000 × 3 months)
Linux Server / VPS Hosting (1 Year)	1,500	Use cheaper VPS or free-tier cloud (or local server)
Network Equipment & LAN Setup	1,000	Minimal setup using existing router & fewer cables
Testing & Debugging	800	Limited but effective testing (basic scenarios)
Documentation & Report Preparation	500	Digital submission instead of printed copies
Miscellaneous / Contingency	1,200	Reduced buffer with better planning

4.2.2 Team Work

The successful completion of this Event Management System project was the result of careful planning, effective teamwork, and clear task delegation. Each team member contributed based on their strengths while maintaining collaborative synergy across all development phases from system architecture to final deployment and documentation. The breakdown of individual contributions is as follows:

Table 4.3: Project Team Work Distribution

Name	ID	Contribution
Md Yusuf Miah	232-15-227	Led overall project coordination and development. Designed and implemented the main Bash script structure, integrated Zenity GUI menus, and handled Samba server configuration (smb.conf). Developed core features like share creation, deletion, and service control (start/stop/restart).
Farzana Begum	232-15-513	Focused on database integration using MariaDB. Created and managed tables for users, shares, and backups. Worked on backend logic for storing and retrieving share data, and handled error checking, debugging, and system stability improvements.
Sunjid Ahmed Siyem	232-15-728	Developed file management functionalities including write, upload, view, edit, and delete operations within shared directories. Assisted in designing user interaction flow using Zenity dialogs and ensured smooth client-side usability.
Mirza Md. Mahmudunnobi Babu	232-15-709	Implemented advanced system features such as Samba user management (add/delete/list users), access control (public/private shares), and service monitoring. Also worked on system performance tuning and secure configuration practices.

Suchona Roy Nishita	232-15-866	Designed and implemented the automated backup system using inotifywait and tar. Managed backup storage, restore functionality, and logging backup history in the database. Contributed to documentation, system testing, and overall integration.

Effective planning and adaptive management ensured steady progress and timely delivery of the project. Continuous monitoring and iterative development helped maintain quality and meet all technical goals.

4.3 Complex Engineering Problem

The Samba Server Manager project addressed challenges in efficiently managing shared files and folders while ensuring seamless, real-time access for multiple users. Optimizing server performance and permission handling was essential to maintain scalability, security, and responsiveness across the network.

4.3.1 Mapping of Program Outcome

In this section, we provide a mapping of our Samba Server Manager project with targeted Program Outcomes (POs) by addressing complex engineering and computing challenges.

CO1 justifies PO1 by

Configuring the Samba Server with proper directory structures, user roles, and access permissions. This required applying core computing and OS knowledge related to file systems, authentication, and network protocols, directly aligning with PO1.

CO2 justifies PO3 by

Implementing modular and reusable configuration files (smb.conf) and scripts for user and group management. These components solve real-world file sharing and access problems efficiently, demonstrating the ability to design and develop effective system-level solutions — fulfilling PO3.

CO3 justifies PO5 by

Using advanced server management tools and optimization techniques, such as monitoring network performance, testing cross-platform access, and tuning permissions for security and efficiency. This demonstrates practical use of modern OS tools and system administration techniques, supporting PO5.

Table 4.4: Justification of Program Outcomes

PO's	Justification
PO1	Apply engineering knowledge in computing
PO3	Design and develop software solutions.
PO5	Use modern tools and techniques.

4.3.2 Complex Problem Solving

In this section, the Samba Server project is mapped to problem-solving categories based on Table 4.5, with rationale for each mapping. For P1 (Depth of Knowledge), an additional mapping for Knowledge Profile is included to highlight its relevance.

CO1 covers the following EP attributes:

- **EP1: Depth of Knowledge Required**
  Understanding fundamental OS and network concepts (like file systems, permissions, user roles, and network protocols) and how they apply to real-world file sharing requires in-depth theoretical knowledge.
- **EP4: Familiarity of Issues**
  Managing a Samba Server involves uncommon issues such as handling permission conflicts, cross-platform access, and secure authentication — topics not routinely encountered outside system administration.

CO2 covers the following EP attributes:

- **EP1: Depth of Knowledge Required**
  Configuring Samba effectively requires knowledge of shared directories, role-based access, and network protocols, which goes beyond basic OS understanding.
- **EP2: Range of Conflicting Requirements**
  Complex problems may arise from competing requirements, such as providing access while maintaining security, requiring abstract thinking and careful design.
- **EP7: Interdependence**
  Server management involves interdependent tasks, such as user creation, folder permission setup, and network access configuration, all requiring coordination across multiple components.

CO3 covers EP4:
Security-related server concerns, such as unauthorized access, privilege escalation, and network vulnerabilities, involve specialized knowledge requiring unique problem-solving techniques.

Table 4.5: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✓	✓		✓			✓

4.3.3 Engineering Activities

In this section, the project will be mapped to the relevant engineering activities based on Table 4.6, with a rationale provided for each mapping.

CO2 covers EA3: SQL query optimization is essential for modern-day applications as it ensures faster response time, reduced server load, and efficient use of resources. Optimizing queries enhances the performance of services in real-time systems, especially when handling large-scale data or high traffic.

Table 4.6: Mapping with complex engineering activities.

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity
		✓		

Chapter 5

Conclusion

This chapter presents a comprehensive summary of the Samba Server Manager system, evaluates its performance and effectiveness, identifies current limitations, and outlines potential areas for future enhancement. The project demonstrates how a traditional Samba Server can be transformed into an automated, intelligent, and user-friendly management system by integrating modern system-level features such as graphical interfaces, monitoring, backup, and database tracking.

5.1 Summary

The Samba Server Manager project successfully achieved its primary objectives by developing a complete and automated Samba Server management system. The system integrates secure file and printer sharing, role-based access control, real-time monitoring, automated backup, and database-driven tracking into a unified platform.

Unlike traditional Samba setups, this system significantly reduces manual configuration efforts by introducing automation through Bash scripting and a GUI-based interface using Zenity. Users can easily perform administrative tasks such as creating shared directories, managing users, controlling services, and handling file operations without requiring advanced command-line knowledge.

The system ensures secure and efficient resource management through proper authentication mechanisms and permission controls. The integration of real-time monitoring using inotify and automated backup mechanisms enhances data reliability and protection. Additionally, the use of a MariaDB database enables structured storage and tracking of system activities, improving transparency and maintainability.

Overall, the project demonstrates a practical and effective implementation of operating system and networking concepts, providing a scalable and user-friendly solution suitable for academic institutions and small organizational environments.

5.2 Limitation

Despite its advanced features, the Samba Server Manager system has certain limitations:

- Designed primarily for small to medium-scale environments
- Limited scalability under heavy concurrent user load
- GUI limited to Zenity (no web or mobile interface)
- Advanced security features (encryption, MFA, IDS) not implemented
- Limited performance testing under high network traffic
- Monitoring system lacks advanced analytics and visualization
- No multi-language support

5.3 Future Work

Future development of the Samba Server Manager system may focus on:

Usability Enhancements

- Web-based dashboard (Flask/Django)
- Mobile-friendly interface
- Multi-language support

Performance Improvements

- Optimization for high user load
- Improved file transfer efficiency
- Load balancing support

Security Enhancements

- Data encryption
- Multi-factor authentication (MFA)
- Intrusion detection system

Monitoring & Analytics

- Real-time monitoring dashboard
- Log visualization
- Alert system

Cloud Integration

- Cloud backup (Google Drive, AWS S3)
- Remote management

References

[1] Samba Team, "Samba — Opening Windows to a Wider World," *Samba Official Documentation*, 2024. [Online]. Available: https://www.samba.org/samba/docs/

[2] M. Ts'o and A. Fiddian-Green, "Linux File System Management and Samba Integration," *Linux Journal*, vol. 2019, no. 292, pp. 1–8, 2019.

[3] R. Eckstein, D. Collier-Brown, and P. Kelly, *Using Samba: A File and Print Server for Linux, Unix & Mac OS X*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2007.

[4] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Upper Saddle River, NJ: Pearson Education, 2014.

[5] Python Software Foundation, "subprocess — Subprocess Management," *Python 3 Standard Library Documentation*, 2024. [Online]. Available: https://docs.python.org/3/library/subprocess.html

[6] Ubuntu Community, "Samba File Server," *Ubuntu Server Documentation*, 2023. [Online]. Available: https://ubuntu.com/server/docs/samba-file-server

[7] K. M. Hossain and S. A. Islam, "Implementation of File Sharing System Using Samba on Linux Platform," *International Journal of Computer Science and Network Security (IJCSNS)*, vol. 18, no. 4, pp. 45–50, 2018.

[8] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Upper Saddle River, NJ: Pearson Education, 2018.

[9] M. R. Islam and M. A. Rahman, "A Study on Centralized File Sharing Using Open-Source Tools in Academic Networks," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 11, no. 7, pp. 312–318, 2020.

[10] R. Love, *Linux System Programming: Talking Directly to the Kernel and C Library*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2013.